P1729R4
Text Parsing

Published Proposal, 2024-02-07

This version:
http://wg21.link/P1729R4

Authors:
Elias Kosunen
Victor Zverovich

Audience:
LEWG, SG9, SG16

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper discusses a new text parsing facility to complement the text formatting functionality of `std::format`, proposed in [P0645].

Table of Contents

§ 1. Revision history

§ 1.1. Changes since R3

- Replace `scan_args_for` with `scan_args` and `wscan_args` for consistency with `std::format`.
- Rename `borrowed_ssubrange_t` to `borrowed_tail_subrange_t` partly based on the naming from ranges-v3 (`tail_view`).
- Replace `format_string` with `scan_format_string`, with a `Range` template parameter.
- Enables compile-time checking for compatibility of the source range, and arguments to scan
- Make `[v]scan_result_type` (the return types of `std::scan` and `std::vscan`) exposition only.
- Remove `visit_scan_arg`: follow [P2637] and use `std::variant::visit`, instead.
- Add discussion on `stdin` support, guided by SG9 polls.
- Make encoding errors be errors for strings, instead of garbage-in-garbage-out.
- Add further discussion on field widths.
- Add example as rationale for mandating `forward_range`.

§ 1.2. Changes since R2

- Return a `subrange` from `scan`, instead of just an iterator: discussion in § 4.5 Argument passing, and return type of scan.
- Default `CharT` to `char` in `scanner` for consistency with `formatter` (previously no default for `CharT`).

- Add design discussion about thousands separators in § 4.3.5.1 Design discussion: Thousands separator grouping checking and § 4.3.5.2 Design discussion: Separate flag for thousands separators.
- Add design discussion about additional error information in § 4.6.2 Design discussion: Additional information.
- Add clarification about field width calculation in § 4.3.4 Width and precision.
- Add note about scope at the end of § 2 Introduction.
- Fix/clarify error handling in example § 3.5 Alternative error handling.
- Address SG16 feedback:
  - Add definition of "whitespace", and clarify matching of non-whitespace literal characters, in § 4.2 Format strings.
  - Add section about text encoding § 4.11 Encoding, and an example about handing reading code units § 4.3.8 Type specifiers: CharT.
  - Add example about using locales in § 4.10 Locales.
  - Add potential future extension: § 6.3 Reading code points (or even grapheme clusters?)

§ 1.3. Changes since R1

- Thoroughly describe the design
- Add examples
- Add specification (synopses only)
- Design changes:
  - Return an `expected` containing a `tuple` from `std::scan`, instead of using output parameters
  - Make `std::scan` take a range instead of a `string_view`
  - Remove support for partial successes

§ 2. Introduction

With the introduction of `std::format` [P0645], standard C++ has a convenient, safe, performant, extensible, and elegant facility for text formatting, over `std::ostream` and the `printf`-family of functions. The story is different for simple text parsing: the standard only provides `std::istream` and the `scanf` family, both of which have issues. This asymmetry is also arguably an inconsistency in the standard library.

According to [CODESEARCH], a C and C++ codesearch engine based on the ACTCD19 dataset, there are 389,848 calls to `sprintf` and 87,815 calls to `sscanf` at the time of writing. So although formatted input functions are less popular than their output counterparts, they are still widely used.

The lack of a general-purpose parsing facility based on format strings has been raised in [P1361] in the context of formatting and parsing of dates and times.

This paper proposes adding a symmetric parsing facility, `std::scan`, to complement `std::format`. This facility is based on the same design principles and shares many features with `std::format`.

This facility is not a parser per se, as it is probably not sufficient for parsing something more complicated, e.g. JSON. This is not a parser combinator library. This is intended to be an almost-drop-in replacement for `sscanf`, capable of being a building block for a more complicated parser.

§ 3. Examples

§ 3.1. Basic example

```
if (auto result = std::scan<std::string, int>("answer = 42", "{} = {}")) {
  //                        ~~~~~~~~~~~~~~~~   ~~~~~~~~~~~    ~~~~~~~
  //                          output types        input        format
```

```
  //                                                          string

  const auto& [key, value] = result->values();
  //           ~~~~~~~~~~
  //            scanned
  //            values

  // result is a std::expected<std::scan_result<...>>.
  // result->range() gives an empty range.
  // result->begin() == result->end()
  // key == "answer"
  // value == 42
} else {
  // We would end up here if we had an error.
  std::scan_error error = result.error();
}
```

§ 3.2. Reading multiple values at once

```
auto input = "25 54.32E-1 Thompson 56789 0123";

auto result = std::scan<int, float, string_view, int, float, int>(
  input, "{:d}{:f}{:9}{:2i}{:g}{:o}");

// result is a std::expected, operator-> will throw if it doesn't contain a value
auto [i, x, str, j, y, k] = result->values();

// i == 25
// x == 54.32e-1
// str == "Thompson"
// j == 56
// y == 789.0
// k == 0123
```

§ 3.3. Reading from a range

```
std::string input{"123 456"};
if (auto result = std::scan<int>(std::views::reverse(input), "{}")) {
  // If only a single value is returned, it can be accessed with result->value()
  // result->value() == 654
}
```

§ 3.4. Reading multiple values in a loop

```
std::vector<int> read_values;
std::ranges::forward_range auto range = ...;

auto input = std::ranges::subrange{range};

while (auto result = std::scan<int>(input, "{}")) {
  read_values.push_back(result->value());
  input = result->range();
}
```

§ 3.5. Alternative error handling

```
// Since std::scan returns a std::expected,
// its monadic interface can be used

auto result = std::scan<int>(..., "{}")
  .transform([](auto result) {
    return result.value();
  });
if (!result) {
  // handle error
}
int num = *result;

// With [P2561]:
int num = std::scan<int>(..., "{}").try?.value();
```

§ 3.6. Scanning a user-defined type

```
struct mytype {
  int a{}, b{};
};

// Specialize std::scanner to add support for user-defined types.
// Inherit from std::scanner<std::string> to get format string parsing
// (scanner::parse()) from it.
template <>
struct std::scanner<mytype> : std::scanner<std::string> {
  template <typename Context>
  auto scan(mytype& val, Context& ctx) const
      -> std::expected<typename Context::iterator, std::scan_error> {
    return std::scan<int, int>(ctx.range(), "[{}, {}]")
      .transform([&val](const auto& result) {
        std::tie(val.a, val.b) = result.values();
        return result.begin();
      });
  }
};

auto result = std::scan<mytype>("[123, 456]", "{}");
// result->value().a == 123
// result->value().b == 456
```

§ 4. Design

The new parsing facility is intended to complement the existing C++ I/O streams library, integrate well with the chrono library, and provide an API similar to `std::format`. This section discusses the major features of its design.

§ 4.1. Overview

The main user-facing part of the library described in this paper, is the function template `std::scan`, the input counterpart of `std::format`. The signature of `std::scan` is as follows:

```
template <class... Args, scannable_range<char> Range>
auto scan(Range&& range, scan_format_string<Range, Args...> fmt)
  -> expected<scan_result<ranges::borrowed_tail_subrange_t<Range>, Args...>, scan_error>;
```

```
template <class... Args, scannable_range<wchar_t> Range>
auto scan(Range&& range, wscan_format_string<Range, Args...> fmt)
  -> expected<scan_result<ranges::borrowed_tail_subrange_t<Range>, Args...>, scan_error>;
```

`std::scan` reads values of type `Args...` from the `range` it's given, according to the instructions given to it in the format string, `fmt`. `std::scan` returns a `std::expected`, containing either a `scan_result`, or a `scan_error`. The `scan_result` object contains a `subrange` pointing to the unparsed input, and a `tuple` of `Args...`, containing the scanned values.

§ 4.2. Format strings

As with `printf`, the `scanf` syntax has the advantage of being familiar to many programmers. However, it has similar limitations:

- Many format specifiers like `hh`, `h`, `l`, `j`, etc. are used only to convey type information. They are redundant in type-safe parsing and would unnecessarily complicate specification and parsing.
- There is no standard way to extend the syntax for user-defined types.
- Using `'%'` in a custom format specifier poses difficulties, e.g. for `get_time`-like time parsing.

Therefore, we propose a syntax based on `std::format` and [PARSE]. This syntax employs `'{'` and `'}'` as replacement field delimiters instead of `'%'`. It will provide the following advantages:

- An easy-to-parse mini-language focused on the data format rather than conveying the type information
- Extensibility for user-defined types
- Positional arguments
- Support for both locale-specific and locale-independent parsing (see § 4.10 Locales)
- Consistency with `std::format`.

At the same time, most of the specifiers will remain quite similar to the ones in `scanf`, which can simplify a, possibly automated, migration.

Maintaining similarity with `scanf`, for any literal non-whitespace character in the format string, an identical character is consumed from the input range. For whitespace characters, all available whitespace characters are consumed.

In this proposal, "whitespace" is defined to be the Unicode code points with the Pattern_White_Space property, as defined by UAX #31 (UAX31-R3a). Those code points are currently:

- ASCII whitespace characters (U+0009 to U+000D, U+0020)
- U+0085 (next line)
- U+200E and U+200F (LEFT-TO-RIGHT MARK and RIGHT-TO-LEFT MARK)
- U+2028 and U+2029 (LINE SEPARATOR and PARAGRAPH SEPARATOR)

Unicode defines a lot of different things in the realm of whitespace, all for different kinds of use cases. The Pattern_White_Space-property is chosen for its stability (it's guaranteed to not change), and because its intended use is for classifying things that should be treated as whitespace in machine-readable syntaxes. `std::isspace` is insufficient for usage in a Unicode world, because it only accepts a single code unit as input.

EXAMPLE 1 ¶

```
auto r0 = std::scan<char>("abcd", "ab{}d"); // r0->value() == 'c'

auto r1 = std::scan<string, string>("abc \n def", "{} {}");
const auto& [s1, s2] = r1->values(); // s1 == "abc", s2 == "def"
```

As mentioned above, the format string syntax consists of replacement fields delimited by curly brackets (`{` and `}`). Each of these replacement fields corresponds to a value to be scanned from the input range. The replacement field syntax is quite

similar to `std::format`, as can be seen in the table below. Elements that are in one but not the other are highlighted.

`scan` replacement field syntax

```
replacement-field ::= '{' [arg-id] [':' format-spec] '}'
format-spec       ::= [fill-and-align]

                      [width]
                      ['L'] [type]
fill-and-align    ::= [fill] align
fill              ::= any character other than
                      '{' or '}'
align             ::= one of '<' '>' '^'

width             ::= positive-integer



type              ::= one of
                      'a' 'A'
                      'b' 'B'
                      'c'
                      'd'
                      'e' 'E'
                      'f' 'F'
                      'g' 'G'
                      'o'
                      'p'
                      's'
                      'x' 'X'
                      '?'
                      'i'
                      'u'
```

`format` replacement field syntax

```
replacement-field ::= '{' [arg-id] [':' form
format-spec       ::= [fill-and-align]
                      [sign] ['#'] ['0']
                      [width] [precision]
                      ['L'] [type]
fill-and-align    ::= [fill] align
fill              ::= any character other th
                      '{' or '}'
align             ::= one of '<' '>' '^'
sign              ::= one of '+' '-' ' '
width             ::= positive-integer
                      OR '{' [arg-id] '}'
precision         ::= '.' nonnegative-intege
                      OR '.' '{' [arg-id] '}
type              ::= one of
                      'a' 'A'
                      'b' 'B'
                      'c'
                      'd'
                      'e' 'E'
                      'f' 'F'
                      'g' 'G'
                      'o'
                      'p'
                      's'
                      'x' 'X'
                      '?'
```

§ 4.3. Format string specifiers

Below is a somewhat detailed description of each of the specifiers in a `std::scan` replacement field. This design attempts to maintain decent compatibility with `std::format` whenever practical, while also bringing in some ideas from `scanf`.

§ 4.3.1. Manual indexing

```
replacement-field ::= '{' [arg-id] [':' format-spec] '}'
```

Like `std::format`, `std::scan` supports manual indexing of arguments in format strings. If manual indexing is used, all of the argument indices have to be spelled out. Different from `std::format`, the same index can only be used once.

EXAMPLE 2 ¶

```
auto r = std::scan<int, int, int>("0 1 2", "{1} {0} {2}");
auto [i0, i1, i2] = r->values();
// i0 == 1, i1 == 0, i2 == 2
```

§ 4.3.2. Fill and align

```
fill-and-align  ::= [fill] align
fill            ::= any character other than
                    '{' or '}'
```

```
align           ::= one of '<' '>' '^'
```

The fill and align options are valid for all argument types. The fill character is denoted by the `fill`-option, or if it is absent, the space character `' '`. The fill character can be any single Unicode scalar value. The field width is determined the same way as it is for `std::format`.

If an alignment is specified, the value to be parsed is assumed to be properly aligned with the specified fill character.

If a field width is specified, it will be the maximum number of characters to be consumed from the input range. In that case, if no alignment is specified, the default alignment for the type is considered (see `std::format`).

For the `'^'` alignment, the number of fill characters needs to be the same as if formatted with `std::format`: `floor(n/2)` characters before, `ceil(n/2)` characters after the value, where `n` is the field width. If no field width is specified, an equal number of alignment characters on both sides are assumed.

This spec is compatible with `std::format`, i.e., the same format string (wrt. fill and align) can be used with both `std::format` and `std::scan`, with round-trip semantics.

NOTE: For format type specifiers other than `'c'` (default for `char` and `wchar_t`, can be specified for `basic_string` and `basic_string_view`), leading whitespace is skipped regardless of alignment specifiers.

EXAMPLE 3 ¶

```
auto r0 = std::scan<int>("   42", "{}"); // r0->value() == 42, r0->range() == ""
auto r1 = std::scan<char>("   x", "{}"); // r1->value() == ' ', r1->range() == "  x"
auto r2 = std::scan<char>("x   ", "{}"); // r2->value() == 'x', r2->range() == "   "

auto r3 = std::scan<int>("    42", "{:6}");  // r3->value() == 42, r3->range() == ""
auto r4 = std::scan<char>("x     ", "{:6}"); // r4->value() == 'x', r4->range() == ""

auto r5 = std::scan<int>("***42", "{:*>}");  // r5->value() == 42
auto r6 = std::scan<int>("***42", "{:*>5}"); // r6->value() == 42
auto r7 = std::scan<int>("***42", "{:*>4}"); // r7->value() == 4
auto r8 = std::scan<int>("42", "{:*>}");     // r8->value() == 42
auto r9 = std::scan<int>("42", "{:*>5}");    // ERROR (mismatching field width)

auto rA = std::scan<int>("42***", "{:*<}");  // rA->value() == 42, rA->range() == ""
auto rB = std::scan<int>("42***", "{:*<5}"); // rB->value() == 42, rB->range() == ""
auto rC = std::scan<int>("42***", "{:*<4}"); // rC->value() == 42, rC->range() == "*"
auto rD = std::scan<int>("42", "{:*<}");     // rD->value() == 42
auto rE = std::scan<int>("42", "{:*<5}");    // ERROR (mismatching field width)

auto rF = std::scan<int>("42", "{:*^}");    // rF->value() == 42, rF->range() == ""
auto rG = std::scan<int>("*42*", "{:*^}");  // rG->value() == 42, rG->range() == ""
auto rH = std::scan<int>("*42**", "{:*^}"); // rH->value() == 42, rH->range() == "*"
auto rI = std::scan<int>("**42*", "{:*^}"); // ERROR (not enough fill characters after v

auto rJ = std::scan<int>("**42**", "{:*^6}"); // rJ->value() == 42, rJ->range() == ""
auto rK = std::scan<int>("*42**", "{:*^5}");  // rK->value() == 42, rK->range() == ""
auto rL = std::scan<int>("**42*", "{:*^6}"); // ERROR (not enough fill characters after
auto rM = std::scan<int>("**42*", "{:*^5}"); // ERROR (not enough fill characters after
```

NOTE: This behavior, while compatible with `std::format`, is very complicated, and potentially hard to understand for users. Since `scanf` doesn't support parsing of fill characters this way, it's possible to leave this feature out for v1, and come back to this later: it's not a breaking change to add formatting specifiers that add new behavior.

4.3.3. Sign, `#`, and `0`

```
format-spec ::= ...
                [sign] ['#'] ['0']
                ...
sign        ::= one of '+' '-' ' '
```

These flags would have no effect in `std::scan`, so they are disabled. Signs (both `+` and `-`), base prefixes, trailing decimal points, and leading zeroes are always allowed for arithmetic values. Disabling them would be a bad default for a higher-level facility like `std::scan`, so flags explicitly enabling them are not needed.

> NOTE: This is incompatible with `std::format` format strings.

4.3.4. Width and precision

```
width     ::= positive-integer
            | '{' [arg-id] '}'
precision ::= '.' nonnegative-integer
            | '.' '{' [arg-id] '}'
```

The width specifier is valid for all argument types. The meaning of this specifier somewhat deviates from `std::format`. The width and precision specifiers of it are combined into a single width specifier in `std::scan`. This specifier indicates the expected field width of the value to be scanned, taking into account possible fill characters used for alignment. If no fill characters are expected, it specifies the maximum width for the field.

> To clarify, in `std::format` the width-field provides the minimum, and the precision-field the maximum width for a value. In `std::scan`, the width-field provides the maximum.
>
> ```
> auto str = std::format("{:2}", 123);
> // str == "123"
> // because only the minimum width was set by the format string
>
> auto result = std::scan<int>(str, "{:2}");
> // result->value() == 12
> // result->range() == "3"
> // because the maximum width was set to 2 by the format string
> ```

For compatibility with `std::format`, the width specifier is in *field width units*, which is specified to be 1 per Unicode (extended) grapheme cluster, except some grapheme clusters are 2 ([format.string.std] ¶ 13):

> For a sequence of characters in UTF-8, UTF-16, or UTF-32, an implementation should use as its field width the sum of the field widths of the first code point of each extended grapheme cluster. Extended grapheme clusters are defined by UAX #29 of the Unicode Standard. The following code points have a field width of 2:
>
> - any code point with the East_Asian_Width="W" or East_Asian_Width="F" Derived Extracted Property as described by UAX #44 of the Unicode Standard
> - U+4dc0 – U+4dff (Yijing Hexagram Symbols)
> - U+1f300 – U+1f5ff (Miscellaneous Symbols and Pictographs)
> - U+1f900 – U+1f9ff (Supplemental Symbols and Pictographs)
>
> The field width of all other code points is 1.
>
> For a sequence of characters in neither UTF-8, UTF-16, nor UTF-32, the field width is unspecified.

This essentially maps 1 field width unit = 1 user perceived character. It should be noted, that with this definition, grapheme clusters like emoji have a field width of 2. This behavior is present in `std::format` today, but can potentially be surprising to users.

This meaning for the width specifier is different from `scanf`, where the width means the number of code units to read. This is because the purpose of that specifier in `scanf` is to prevent buffer overflow. Because the current interface of the proposed `std::scan` doesn't allow reading into an user-defined buffer, this isn't a concern.

> Other options can be considered, if compatibility with `std::format` can be set aside. These options include:
>
> - Plain bytes or code units
> - Unicode code points
> - Unicode (extended) grapheme clusters
> - `std::format`-like field width units, except only looking at code points, instead of grapheme clusters
> - Exclusively using UAX #11 (East Asian Width) widths

Specifying the width with another argument, like in `std::format`, is disallowed.

4.3.5. Localized (`L`)

```
format-spec ::= ...
                ['L']
                ...
```

Enables scanning of values in locale-specific forms.

- For integer types, allows for digit group separator characters, equivalent to `numpunct::thousands_sep` of the used locale. If digit group separator characters are used, their grouping must match `numpunct::grouping`.
- For floating-point types, the same as above. In addition, the locale-specific radix separator character is used, from `numpunct::decimal_point`.
- For `bool`, the textual representation uses the appropriate strings from `numpunct::truename` and `numpunct::falsename`.

4.3.5.1. Design discussion: Thousands separator grouping checking

As proposed, when using localized scanning, the grouping of thousands separators in the input must exactly match the value retrieved from `numpunct::grouping`. This behavior is consistent with iostreams. It may, however, be undesirable: it is possible, that the user would supply values with incorrect thousands separator grouping, but that may need not be an error. The number is still unambiguously parseable, with the check for grouping only done after parsing.

EXAMPLE 4 ¶

```
struct custom_numpunct : std::numpunct<char> {
  std::string do_grouping() const override {
    return "\3";
  }

  char do_thousands_sep() const override {
    return ',';
  }
};

auto loc = std::locale(std::locale::classic(), new custom_numpunct);

// As proposed:
// Check grouping, error if invalid
auto r0 = std::scan<int>(loc, "123,45", "{:L}");
// r0.has_value() == false

// ALTERNATIVE:
// Do not check grouping, only skip it
auto r1 = std::scan<int>(loc, "123,45", "{:L}");
// r1.has_value() == true
// r1->value() == 12345

// Current proposed behavior, _somewhat_ consistent with iostreams:
istringstream iss{"123,45"};
iss.imbue(locale(locale::classic(), new custom_numpunct));

int i{};
iss >> i;
// i == 12345
// iss.fail() == !iss == true
```

This highlights a problem with using `std::expected`: we can either have a value, or an error. IOStreams can both return an error, and a value. This issue is also present with range errors with ints and floats, see § 4.6.2 Design discussion: Additional information for more.

§ *4.3.5.2. Design discussion: Separate flag for thousands separators*

It may also be desirable to split up the behavior of skipping and checking of thousands separators from the realm of localization. For example, in the POSIX-extended version of `sscanf`, there's the `'` format specifier, which allows opting-into reading of thousands separators.

When a locale isn't used, a set of options similar to the thousands separator options used with the `en_US` locale (i.e. `,` with `"\3"` grouping). This would enable skipping of thousands separators without involving locale.

EXAMPLE 5 ¶

```
// NOT PROPOSED,
// hypothetical example, with a ' format specifier

auto r = std::scan<int>("123,456", "{:'}");
// r->value() == 123456
```

§ **4.3.6. Type specifiers: strings**

Type	Meaning

none, `s`	Copies from the input until a whitespace character is encountered.
`?`	Copies an escaped string from the input.
`c`	Copies from the input until the field width is exhausted. Does not skip preceding whitespace. Errors, if no field width is provided.

The `s` specifier is consistent with `std::istream` and `std::string`:

```
std::string word;
std::istringstream{"Hello world"} >> word;
// word == "Hello"

auto r = std::scan<string>("Hello world", "{:s}");
// r->value() == "Hello"
```

NOTE: The `c` specifier is consistent with `scanf`, but is not supported for strings by `std::format`.

§ 4.3.7. Type specifiers: integers

Integer values are scanned as if by using `std::from_chars`, except:

- A positive `+` sign and a base prefix are always allowed to be present.
- Preceding whitespace is skipped.

Type	Meaning
`b`, `B`	`from_chars` with base 2. The base prefix is `0b` or `0B`.
`o`	`from_chars` with base 8. For non-zero values, the base prefix is `0`.
`x`, `X`	`from_chars` with base 16. The base prefix is `0x` or `0X`.
`d`	`from_chars` with base 10. No base prefix.
`u`	`from_chars` with base 10. No base prefix. No `-` sign allowed.
`i`	Detect base from a possible prefix, default to decimal.
`c`	Copies a character from the input.
none	Same as `d`

NOTE: The flags `u` and `i` are not supported by `std::format`. These flags are consistent with `scanf`.

§ 4.3.8. Type specifiers: `CharT`

Type	Meaning
none, `c`	Copies a character from the input.
`b`, `B`, `d`, `i`, `o`, `u`, `x`, `X`	Same as for integers.
`?`	Copies an escaped character from the input.

EXAMPLE 6 ¶

This is not encoding or Unicode-aware. Reading a `CharT` with the `c` type specifier will just read a single code unit of type `CharT`. This can lead to invalid encoding in the scanned values.

```
// As proposed:
// U+12345 is 0xF0 0x92 0x8D 0x85 in UTF-8
auto r = std::scan<char, std::string>("\u{12345}", "{}{}");
auto& [ch, str] = r->values();
// ch == '\xF0'
// str == "\x92\x8d\x85" (invalid utf-8)

// This is the same behavior as with iostreams today
```

4.3.9. Type specifiers: `bool`

Type	Meaning
`s`	Allows for textual representation, i.e. `true` or `false`
`b`, `B`, `d`, `i`, `o`, `u`, `x`, `X`	Allows for integral representation, i.e. `0` or `1`
none	Allows for both textual and integral representation: i.e. `true`, `1`, `false`, or `0`.

4.3.10. Type specifiers: floating-point types

Similar to integer types, floating-point values are scanned as if by using `std::from_chars`, except:

- A positive `+` sign is always allowed to be present.
- Preceding whitespace is skipped.

Type	Meaning
`a`, `A`	`from_chars` with `chars_format::hex`, with `0x`/`0X`-prefix allowed.
`e`, `E`	`from_chars` with `chars_format::scientific`.
`f`, `F`	`from_chars` with `chars_format::fixed`.
`g`, `G`	`from_chars` with `chars_format::general`.
none	`from_chars` with `chars_format::general | chars_format::hex`, with `0x`/`0X`-prefix allowed.

4.4. Ranges

We propose, that `std::scan` would take a range as its input. This range should satisfy the requirements of `std::ranges::forward_range` to enable look-ahead, which is necessary for parsing.

```
template <class Range, class CharT>
concept scannable_range =
  ranges::forward_range<Range> && same_as<ranges::range_value_t<Range>, CharT>;
```

For a range to be a `scannable_range`, its character type (range `value_type`, code unit type) needs to also be correct, i.e. it needs to match the character type of the format string. Mixing and matching character types between the input range and the format string is not supported.

EXAMPLE 7 ¶

```
scan<int>("42", "{}");   // OK
scan<int>(L"42", L"{}"); // OK
scan<int>(L"42", "{}");  // Error: wchar_t\[N] is not a scannable_range<char>
```

It should be noted, that standard range facilities related to iostreams, namely `std::istreambuf_iterator`, model `input_iterator`. Thus, they can't be used with `std::scan`, and therefore, for example, `stdin`, can't be read directly using `std::scan`. The reference implementation deals with this by providing a range type, that wraps a `std::basic_istreambuf`, and provides a `forward_range`-compatible interface to it. At this point, this is deemed out of scope for this proposal.

As mentioned above, `forward_range`s are needed to support proper lookahead and rollback. For example, when reading an `int` with the `i` format specifier (detect base from prefix), whether a character is part of the `int` can't be determined before reading past it.

EXAMPLE 8 ¶

```
// Hex value "0xf"
auto r1 = std::scan<int>("0xf", "{:i}");
// r1->value() == 0xf
// r1->range().empty() == true

// (Octal) value "0", with "xg" left over
auto r2 = std::scan<int>("0xg", "{:i}");
// r2->value() == 0
// r2->range() == "xg"
```

This behavior is different from `scanf`.

The same behavior can be observed with floating-point values, when using exponents: whether `1e+X` is parsed as a number, or as `1` with the rest left over, depends on whether `X` is a valid exponent. For user-defined types, arbitrarily-long look-/rollback can be required.

§ 4.5. Argument passing, and return type of `scan`

In an earlier revision of this paper, output parameters were used to return the scanned values from `std::scan`. In this revision, we propose returning the values instead, wrapped in an `expected`.

```
// R2 (current)
auto result = std::scan<int>(input, "{}");
auto [i] = result->values();
// or:
auto i = result->value();

// R1 (previous)
int i;
auto result = std::scan(input, "{}", i);
```

The rationale behind this change is as follows:

- It was easy to accidentally use uninitialized values (as evident by the example above). In this revision, the values can only be accessed when the operation is successful.
- Modern C++ API design principles favor return values over output parameters.
- The earlier design was conceived at a time, when C++17 support and usage wasn't as prevalent as it is today. Back then, the only way to use a return-value API was through `std::tie`, which wasn't ergonomic.
- Previously, there were real performance implications when using complicated tuples, both at compile-time and runtime. These concerns have since been alleviated, as compiler technology has improved.

It should be noted, that not using output parameters removes a channel for user customization. For example, [FMT] uses `fmt::arg` to specify named arguments. The same isn't directly possible here.

The return type of `scan`, `scan_result`, contains a `subrange` over the unparsed input. With this, a new type alias is introduced, `ranges::borrowed_tail_subrange_t`, that is defined as follows:

```
template <typename R>
using borrowed_tail_subrange_t = std::conditional_t<
  ranges::borrowed_range<R>,
  ranges::subrange<ranges::iterator_t<R>, ranges::sentinel_t<R>>,
  ranges::dangling>;
```

Compare this with `borrowed_subrange_t`, which is defined as `ranges::subrange<ranges::iterator_t<R>, ranges::iterator_t<R>>`, when the range models `borrowed_range`.

This is novel in the ranges space: previously all algorithms have either returned an iterator, or a subrange of two iterators. We believe that `std::scan` warrants a diversion: if (for `I = ranges::iterator_t<R>` and `S = ranges::sentinel_t<R>`) `std::random_access_iterator<I> || std::sized_sentinel_for<I> || std::assignable_from<I&, S>` is `false`, `std::scan` will need to go through the rest of the input, in order to get the end iterator to return. A superior alternative is to simply return the sentinel, since that's always correct (the leftover range always has the same end as the source range) and requires no additional computation.

See this StackOverflow answer by Barry Revzin for more context: [BARRY-SO-ANSWER].

4.5.1. Design alternatives

As proposed, `std::scan` returns an `expected`, containing either an iterator and a tuple, or a `scan_error`.

An alternative could be returning a `tuple`, with a result object as its first (0th) element, and the parsed values occupying the rest. This would enable neat usage of structured bindings:

```
// NOT PROPOSED, design alternative
auto [r, i] = std::scan<int>("42", "{}");
```

However, there are two possible issues with this design:

1. It's easy to accidentally skip checking whether the operation succeeded, and access the scanned values regardless. This could be a potential security issue (even though the values would always be at least value-initialized, not default-initialized). Returning an expected forces checking for success.
2. The numbering of the elements in the returned tuple would be off-by-one compared to the indexing used in format strings:

   ```
   auto r = std::scan<int>("42", "{0}");
   // std::get<0>(r) refers to the result object
   // std::get<1>(r) refers to {0}
   ```

For the same reason as enumerated in 2. above, the `scan_result` type as proposed doesn't follow the tuple protocol, so that structured bindings can't be used with it:

```
// NOT PROPOSED
auto result = std::scan<int>("42", "{0}");
// std::get<0>(*result) would refer to the iterator
// std::get<1>(*result) would refer to {0}
```

4.6. Error handling

Contrasting with `std::format`, this proposed library communicates errors with return values, instead of throwing exceptions. This is because error conditions are expected to be much more frequent when parsing user input, as opposed to text formatting. With the introduction of `std::expected`, error handling using return values is also more ergonomic than before, and it provides a vocabulary type we can use here, instead of designing something novel.

`std::scan_error` holds an enumerated error code value, and a message string. The message is used in the same way as the message in `std::exception`: it gives more details about the error, but its contents are unspecified.

```
// Not a specification, just exposition
class scan_error {
public:
  enum code_type {
    good,

    // EOF:
    // tried to read from an empty range,
    // or the input ended unexpectedly.
```

```
    // Naming alternative: end_of_input
    end_of_range,

    invalid_format_string,

    invalid_scanned_value,

    value_out_of_range
  };

  constexpr scan_error() = default;
  constexpr scan_error(code_type, const char*);

  constexpr explicit operator bool() const noexcept;

  constexpr code_type code() const noexcept;
  constexpr const char* msg() const;
};
```

4.6.1. Design discussion: Essence of `std::scan_error`

The reason why we propose adding the type `std::scan_error` instead of just using `std::errc` is, that we want to avoid losing information. The enumerators of `std::errc` are insufficient for this use, as evident by the table below: there are no clear one-to-one mappings between `scan_error::code_type` and `std::errc`, but `std::errc::invalid_argument` would need to cover a lot of cases.

The `const char*` in `scan_error` is extremely useful for user code, for use in logging and debugging. Even with the `scan_error::code_type` enumerators, more information is often needed, to isolate any possible problem.

Possible mappings from `scan_error::code_type` to `std::errc` could be:

`scan_error::code_type`	`errc`
`scan_error::good`	`std::errc{}`
`scan_error::end_of_range`	`std::errc::invalid_argument`
`scan_error::invalid_format_string`	
`scan_error::invalid_scanned_value`	
`scan_error::value_out_of_range`	`std::errc::result_out_of_range`

There are multiple dimensions of design decisions to be done here:

1. **Should `scan_error` use a custom enumeration?**
    1. **Yes.** (currently proposed, our preference)
    2. **No, use `std::errc`.** Loses precision in error codes
2. **Should `scan_error` contain a message?**
    1. **Yes, a `const char*`.** (currently proposed, weak preference)
    2. **Yes, a `std::string`.** Potentially more expensive.
    3. **No.** Worse user experience for loss of diagnostic information

4.6.2. Design discussion: Additional information

Only having `value_out_of_range` does not give a way to differentiate between different kinds of out-of-range errors, like overflowing (absolute value too large, either positive or negative), or underflowing (value not representable, between zero and the smallest subnormal).

Both `std::istream` (through `std::num_get`), and the `std::strto*` family of functions support differentiating between differentiating between different kinds of overflow and underflow, through the magnitude of the returned value. `std::from_chars` currently does not (see [LWG3081]). `std::scanf` does not, either.

EXAMPLE 9 ¶

```
// larger than INT32_MAX
std::string source{"99999999999999999999999999999"};

{
  std::istringstream iss{source};
  int i{};
  iss >> i;
  // iss.fail() == true
  // i == INT32_MAX
}

{
  // (assuming sizeof(long) == 4)
  auto i = std::strtol(source.c_str(), nullptr, 10);
  // i == LONG_MAX
  // errno == ERANGE
}

{
  int i{};
  auto [ec, ptr] = std::from_chars(source.data(), source.data() + source.size(), i);
  // ec == std::errc::result_out_of_range
  // i == 0 (!)
}

{
  int i{};
  auto r = std::sscanf(source.c_str(), "%d", &i);
  // r == 1  (?)
  // i == -1 (?)
  // errno == ERANGE
}
```

This is predicated on an issue with using `std::expected`: we can only ever either return an error, or a value. Those aforementioned facilities can both return an error code, while simultaneously communicating additional information about possible errors through the scanned value.

Nevertheless, there's a simple reason for using `std::expected`: it prevents user errors. Because an `expected` can indeed only hold either a value or an error, there's never a situation where an user accidentally forgets to check for an error, and mistakenly uses the scanned value directly instead:

EXAMPLE 10 ¶

```
int i{};
std::cin >> i;
// We would need to check std::cin.operator bool() first,
// to determine whether `i` was successfully read:
// that's very easy to forget

auto r = std::scan<int>(..., "{}");
int i = r->value();
//        ^
//      dereference
// does not allow for accidentally accessing the value if we had an error
```

It's a tradeoff. Either we allow for an additional avenue for error reporting through the scanned value, or we use `expected` to prevent reading the values during an error. Currently, this paper propses doing the latter.

§ 4.7. Binary footprint and type erasure

We propose using a type erasure technique to reduce the per-call binary code size. The scanning function that uses variadic templates can be implemented as a small inline wrapper around its non-variadic counterpart:

```
template<scannable_range<char> Range>
auto vscan(Range&& range, string_view fmt, scan_args args)
  -> expected<ranges::borrowed_tail_subrange_t<Range>, scan_error>;

template <typename... Args, scannable_range<char> SourceRange>
auto scan(SourceRange&& source, scan_format_string<Range, Args...> format)
    -> expected<scan_result<ranges::borrowed_tail_subrange_t<SourceRange>, Args...>, scan_e
  auto args = make_scan_args<SourceRange, Args...>();
  auto result = vscan(std::forward<SourceRange>(range), format, args);
  return make_scan_result(std::move(result), std::move(args));
}
```

As shown in [P0645] this dramatically reduces binary code size, which will make `scan` comparable to `scanf` on this metric.

`make_scan_args` type erases the arguments that are to be scanned. This is similar to `std::make_format_args`, used with `std::format`.

> NOTE: This implementation of `std::scan` is more complicated compared to `std::format`, which can be described as a one-liner calling `std::vformat`. This is because the *`scan-arg-store`* returned by `make_scan_args` needs to outlive the call to `vscan`, and then be converted to a `tuple` and returned from `scan`. Whereas with `std::format`, the *`format-arg-store`* returned by `std::make_format_args` is immediately consumed by `std::vformat`, and not used elsewhere.

§ 4.8. Safety

`scanf` is arguably more unsafe than `printf` because `__attribute__((format(scanf, ...)))` ([ATTR]) implemented by GCC and Clang doesn't catch the whole class of buffer overflow bugs, e.g.

```
char s[10];
std::sscanf(input, "%s", s); // s may overflow.
```

Specifying the maximum length in the format string above solves the issue but is error-prone, especially since one has to account for the terminating null.

Unlike `scanf`, the proposed facility relies on variadic templates instead of the mechanism provided by `<cstdarg>`. The type information is captured automatically and passed to scanners, guaranteeing type safety and making many of the `scanf` specifiers redundant (see § 4.2 Format strings). Memory management is automatic to prevent buffer overflow errors.

§ 4.9. Extensibility

We propose an extension API for user-defined types similar to `std::formatter`, used with `std::format`. It separates format string processing and parsing, enabling compile-time format string checks, and allows extending the format specification language for user types. It enables scanning of user-defined types.

```
auto r = scan<tm>(input, "Date: {0:%Y-%m-%d}");
```

This is done by providing a specialization of `scanner` for `tm`:

```
template <>
struct scanner<tm> {
  constexpr auto parse(scan_parse_context& ctx)
```

```
    -> expected<scan_parse_context::iterator, scan_error>;

  template <class ScanContext>
  auto scan(tm& t, ScanContext& ctx) const
    -> expected<typename ScanContext::iterator, scan_error>;
};
```

The `scanner<tm>::parse` function parses the `format-spec` portion of the format string corresponding to the current argument, and `scanner<tm>::scan` parses the input range `ctx.range()` and stores the result in `t`.

An implementation of `scanner<T>::scan` can potentially use the istream extraction `operator>>` for user-defined type `T`, if available.

§ 4.10. Locales

As pointed out in [N4412]:

> There are a number of communications protocol frameworks in use that employ text-based representations of data, for example XML and JSON. The text is machine-generated and machine-read and should not depend on or consider the locales at either end.

To address this, `std::format` provided control over the use of locales. We propose doing the same for the current facility by performing locale-independent parsing by default and designating separate format specifiers for locale-specific ones. In particular, locale-specific behavior can be opted into by using the `L` format specifier, and supplying a `std::locale` object.

EXAMPLE 11 ¶

```
std::locale::global(std::locale::classic());

// {} uses no locale
// {:L} uses the global locale
auto r0 = std::scan<double, double>("1.23 4.56", "{} {:L}");
// r0->values(): (1.23, 4.56)

// {} uses no locale
// {:L} uses the supplied locale
auto r1 = std::scan<double, double>(std::locale{"fi_FI"}, "1.23 4,56", "{} {:L}");
// r1->values(): (1.23, 4.56)
```

§ 4.11. Encoding

In a similar manner as with `std::format`, input given to `std::scan` is assumed to be in the (ordinary/wide) literal encoding.

If an error in encoding is encountered while reading a value of a string type (`basic_string`, `basic_string_view`), an `invalid_scanned_value` error is returned. For other types, the reading is stopped, as the parser can't parse a numeric value from something that isn't digits.

EXAMPLE 12 ¶

```
// Invalid UTF-8
auto r = std::scan<std::string>("a\xc3 ", "{}");
// r == false
// r->error() == std::scan_error::invalid_scanner_value

auto r2 = std::scan<int>("1\xc3 ", "{}");
// r2 == true
// r2->value() == 1
// r2->range() == "\xc3 "
```

Reading raw bytes (not in the literal encoding) into a `string` isn't directly supported. This can be achieved either with simpler range algorithms already in the standard, or by using a custom type or scanner.

§ 4.12. Performance

The API allows efficient implementation that minimizes virtual function calls and dynamic memory allocations, and avoids unnecessary copies. In particular, since it doesn't need to guarantee the lifetime of the input across multiple function calls, `scan` can take `string_view` avoiding an extra string copy compared to `std::istringstream`. Since, in the default case, it also doesn't deal with locales, it can internally use something like `std::from_chars`.

We can also avoid unnecessary copies required by `scanf` when parsing strings, e.g.

```
auto r = std::scan<std::string_view, int>("answer = 42", "{} = {}");
```

Because the format strings are checked at compile time, while being aware of the exact types to scan, and the source range type, it's possible to check at compile time, whether scanning a `string_view` would dangle, or if it'S possible at all (reading from a non-`contiguous_range`).

§ 4.13. Integration with chrono

The proposed facility can be integrated with `std::chrono::parse` ([P0355]) via the extension mechanism, similarly to the integration between chrono and text formatting proposed in [P1361]. This will improve consistency between parsing and formatting, make parsing multiple objects easier, and allow avoiding dynamic memory allocations without resolving to the deprecated `strstream`.

Before:

```
std::istringstream is("start = 10:30");
std::string key;
char sep;
std::chrono::seconds time;
is >> key >> sep >> std::chrono::parse("%H:%M", time);
```

After:

```
auto result = std::scan<std::string, std::chrono::seconds>("start = 10:30", "{0} = {1:%H:%M
const auto& [key, time] = result->values();
```

Note that the `scan` version additionally validates the separator.

§ 4.14. Impact on existing code

The proposed API is defined in a new header and should have no impact on existing code.

§ 5. Existing work

[SCNLIB] is a C++ library that, among other things, provides an interface similar to the one described in this paper. As of the publication of this paper, the latest release (v2.0.0) of [SCNLIB] is the reference implementation of this proposal.

[FMT] has a prototype implementation of an earlier version of the proposal.

§ 6. Future extensions

To keep the scope of this paper somewhat manageable, we've chosen to only include functionality we consider fundamental. This leaves the design space open for future extensions and other proposals. However, we are not categorically against exploring this design space, if it is deemed critical for v1.

All of the possible future extensions described below are implemented in [SCNLIB].

§ 6.1. Integration with `stdio`

In the SG9 meeting in Kona (11/2023), it was polled, that:

> SG9 feels that it essential for std::scan to be useable with stdin and cin (and the paper would be incomplete without this feature).
>
SF	F	N	A	SA
0	5	1	3	0

We've decided to follow the route of `std::format` + `std::print`, i.e. to not complicate and bloat this paper further by involving I/O. This is still an important avenue of future expansion, and the library proposed in this paper is designed and specified in such a way as to easily allow that expansion.

[SCNLIB] implements this by providing a function, `scn::input`, for interfacing with `stdin`, and by allowing passing in `FILE*`s as input to `scn::scan`, in addition to `scannable_range`s.

§ 6.2. `scanf`-like `[character set]` matching

`scanf` supports the `[` format specifier, which allows for matching for a set of accepted characters. Unfortunately, because some of the syntax for specifying that set is implementation-defined, the utility of this functionality is hampered. Properly specified, this could be useful.

EXAMPLE 13 ¶

```
auto r = scan<string>("abc123", "{:[a-zA-Z]}"); // r->value() == "abc", r->range() == "1
// Compare with:
char buf\[N];
sscanf("abc123", "%[a-zA-Z]", buf);

// ...

auto _ = scan<string>(..., "{:[^\n]}"); // match until newline
```

It should be noted, that while the syntax is quite similar, this is not a regular expression. This syntax is intentionally way more limited, as is meant for simple character matching.

[SCNLIB] implements this syntax, providing support for matching single characters/code points (`{:[abc]}`) and code point ranges (`{:[a-z]}`). Full regex matching is also supported with `{:/.../}`.

§ 6.3. Reading code points (or even grapheme clusters?)

`char32_t` in nowadays the type denoting a Unicode code point. Reading individual code points, or even Unicode grapheme clusters, could be a useful feature. Currently, this proposal only supports reading of individual code units (`char` or `wchar_t`).

[SCNLIB] supports reading Unicode code points with `char32_t`.

§ 6.4. Reading strings and chars of different width

In C++, we have character types other than `char` and `wchar_t`, too: namely `char8_t`, `char16_t`, and `char32_t`. Currently, this proposal only supports reading strings with the same character type as the input range, and reading `wchar_t` characters from narrow `char`-oriented input ranges, as does `std::format`. `scanf` somewhat supports this with the `l`-flag (and the absence of one in `wscanf`). Providing support for reading differently-encoded strings could be useful.

EXAMPLE 14 ¶

```
// Currently supported:
auto r0 = scan<wchar_t>("abc", "{}");

// Not supported:
auto r1 = scan<char>(L"abc", L"{}");
auto r2 =
  scan<string, wstring, u8string, u16string, u32string>("abc def ghi jkl mno", "{} {} {}
auto r3 =
  scan<string, wstring, u8string, u16string, u32string>(L"abc def ghi jkl mno", L"{} {}
```

§ 6.5. Scanning of ranges

Introduced in [P2286] for `std::format`, enabling the user to use `std::scan` to scan ranges, could be useful.

§ 6.6. Default values for scanned values

Currently, the values returned by `std::scan` are value-constructed, and assigned over if a value is read successfully. It may be useful to be able to provide an initial value different from a value-constructed one, for example, for preallocating a `string`, and possibly reusing it:

EXAMPLE 15 ¶

```
string str;
str.reserve(n);
auto r0 = scan<string>(..., "{}", {std::move(str)});
// ...
r0->value().clear();
auto r1 = scan<string>(..., "{}", {std::move(r0->value())});
```

This same facility could be also used for additional user customization, as pointed out in § 4.5 Argument passing, and return type of scan.

§ 6.7. Assignment suppression / discarding values

`scanf` supports discarding scanned values with the `*` specifier in the format string. [SCNLIB] provides similar functionality through a special type, `scn::discard`:

EXAMPLE 16

```
int i;
scanf("%*d", &i);

auto r = scn::scan<scn::discard<int>>(..., "{}");
auto [_] = r->values();
```

7. Specification

At this point, only the synopses are provided.

Note the similarity with [P0645] (`std::format`) in some parts.

The changes to the wording include additions to the header `<ranges>`, and a new header, `<scan>`.

7.1. Modify "Header `<ranges>` synopsis" [ranges.syn]

```
#include <compare>
#include <initializer_list>
#include <iterator>

namespace std::ranges {
  // ...

  template<range R>
    using borrowed_iterator_t = see below;   // freestanding

  template<range R>
    using borrowed_subrange_t = see below;   // freestanding

  template<range R>
    using borrowed_tail_subrange_t = see below;  // freestanding

  // ...
}
```

7.2. Modify "Dangling iterator handling", paragraph 3 [range.dangling]

For a type `R` that models `range`:

- if `R` models `borrowed_range`, then `borrowed_iterator_t<R>` denotes `iterator_t<R>`, ~~and `borrowed_subrange_t<R>` denotes `subrange<iterator_t<R>>`~~; `borrowed_subrange_t<R>` denotes `subrange<iterator_t<R>>`, and `borrowed_tail_subrange_t<R>` denotes `subrange<iterator_t<R>, sentinel_t<R>>`;
- otherwise, ~~both `borrowed_iterator_t<R>` and `borrowed_subrange_t<R>` denote `dangling`~~. `borrowed_iterator_t<R>`, `borrowed_subrange_t<R>`, and `borrowed_tail_subrange_t<R>` all denote `dangling`.

7.3. Header `<scan>` synopsis

```
namespace std {
  template<class charT, class Range, class... Args>
    struct basic_scan_format_string;

  template<class Range, class... Args>
```

```
  using scan_format_string =
    basic_scan_format_string<char,
                             type_identity_t<Range>,
                             type_identity_t<Args>...>;
template<class Range, class... Args>
  using wscan_format_string =
    basic_scan_format_string<wchar_t,
                             type_identity_t<Range>,
                             type_identity_t<Args>...>;

class scan_error;

template<class Range, class... Args>
  class scan_result;

template<class Range, class CharT>
  concept scannable_range =
    ranges::forward_range<Range> &&
    same_as<ranges::range_value_t<Range>, CharT>;

template<class Range, class... Args>
  using scan-result-type = expected<
    scan_result<ranges::borrowed_tail_subrange_t<Range>, Args...>,
    scan_error>; // exposition only

template<class... Args, scannable_range<char> Range>
  scan-result-type<Range, Args...> scan(Range&& range,
                                        scan_format_string<Range, Args...> fmt);

template<class... Args, scannable_range<wchar_t> Range>
  scan-result-type<Range, Args...> scan(Range&& range,
                                        wscan_format_string<Range, Args...> fmt);

template<class... Args, scannable_range<char> Range>
  scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                        scan_format_string<Range, Args...> fmt);

template <class... Args, scannable_range<wchar_t> Range>
  scan-result-type<Range, Args...> scan(const locale& loc, Range&& range,
                                        wscan_format_string<Range, Args...> fmt);

template<class Range, class charT> class basic_scan_context;
using scan_context = basic_scan_context<unspecified, char>;
using wscan_context = basic_scan_context<unspecified, wchar_t>;

template<class Context> class basic_scan_args;
using scan_args = basic_scan_args<scan_context>;
using wscan_args = basic_scan_args<wscan_context>;

template<class Range>
  using vscan-result-type = expected<
    ranges::borrowed_tail_subrange_t<Range>,
    scan_error>; // exposition only

template<scannable_range<char> Range>
  vscan-result-type<Range> vscan(Range&& range, string_view fmt, scan_args args);

template<scannable_range<wchar_t> Range>
  vscan-result-type<Range> vscan(Range&& range, wstring_view fmt, wscan_args args);

template<scannable_range<char> Range>
  vscan-result-type<Range> vscan(const locale& loc,
                                 Range&& range,
                                 string_view fmt,
                                 scan_args args);
```

```
  template<scannable_range<wchar_t> Range>
    vscan-result-type<Range> vscan(const locale& loc,
                                   Range&& range,
                                   wstring_view fmt,
                                   wscan_args args);

  template<class T, class CharT = char>
    struct scanner;

  template<class T, class CharT>
    concept scannable = see below;

  template<class CharT>
    using basic_scan_parse_context = basic_format_parse_context<CharT>;

  using scan_parse_context = basic_scan_parse_context<char>;
  using wscan_parse_context = basic_scan_parse_context<wchar_t>;

  template<class Context>
    class basic_scan_arg;

  template<class Context, class... Args>
    class scan-arg-store; // exposition only

  template<class Range, class... Args>
    constexpr see below make_scan_args();

  template<class Range, class Context, class... Args>
    expected<scan_result<Range, Args...>, scan_error>
      make_scan_result(expected<Range, scan_error>&& source,
                       scan-arg-store<Context, Args...>&& args);
}
```

§ 7.4. Class `scan_error` synopsis

```
namespace std {
  class scan_error {
  public:
    enum code_type {
      good,
      end_of_range,
      invalid_format_string,
      invalid_scanned_value,
      value_out_of_range
    };

    constexpr scan_error() = default;
    constexpr scan_error(code_type error_code, const char* message);

    constexpr explicit operator bool() const noexcept;

    constexpr code_type code() const noexcept;
    constexpr const char* msg() const;

  private:
    code_type code_;      // exposition only
    const char* message_; // exposition only
  };
}
```

§ 7.5. Class template `scan_result` synopsis

```
namespace std {
  template<class Range, class... Args>
  class scan_result {
  public:
    using range_type = Range;

    constexpr scan_result() = default;
    constexpr ~scan_result() = default;

    constexpr scan_result(range_type r, tuple<Args...>&& values);

    template<class OtherR, class... OtherArgs>
      constexpr explicit(see below) scan_result(OtherR&& it, tuple<OtherArgs...>&& values);

    constexpr scan_result(const scan_result&) = default;
    template<class OtherR, class... OtherArgs>
      constexpr explicit(see below) scan_result(const scan_result<OtherR, OtherArgs...>& ot

    constexpr scan_result(scan_result&&) = default;
    template<class OtherR, class... OtherArgs>
      constexpr explicit(see below) scan_result(scan_result<OtherR, OtherArgs...>&& other);

    constexpr scan_result& operator=(const scan_result&) = default;
    template<class OtherR, class... OtherArgs>
      constexpr scan_result& operator=(const scan_result<OtherR, OtherArgs...>& other);

    constexpr scan_result& operator=(scan_result&&) = default;
    template<class OtherR, class... OtherArgs>
      constexpr scan_result& operator=(scan_result<OtherR, OtherArgs...>&& other);

    constexpr range_type range() const;

    constexpr see below begin() const;
    constexpr see below end() const;

    template<class Self>
      constexpr auto&& values(this Self&&);

    template<class Self>
      requires sizeof...(Args) == 1
      constexpr auto&& value(this Self&&);

  private:
    range_type range_;      // exposition only
    tuple<Args...> values_; // exposition only
  };
}
```

§ 7.6. Class template `basic_scan_format_string` synopsis

```
namespace std {
  template<class charT, class Range, class... Args>
  struct basic_scan_format_string {
  private:
    basic_string_view<charT> str;  // exposition only

  public:
    template<class T> consteval basic_scan_format_string(const T& s);
```

```
    constexpr basic_string_view<charT> get() const noexcept { return str; }
  };
}
```

§ 7.7. Class template `basic_scan_context` synopsis

```
namespace std {
  template<class Range, class CharT>
  class basic_scan_context {
  public:
    using char_type = CharT;
    using range_type = Range;
    using iterator = ranges::iterator_t<range_type>;
    using sentinel = ranges::sentinel_t<range_type>;
    template<class T> using scanner_type = scanner<T, char_type>;

    constexpr basic_scan_arg<basic_scan_context> arg(size_t id) const noexcept;
    std::locale locale();

    constexpr iterator begin() const;
    constexpr sentinel end() const;
    constexpr range_type range() const;
    constexpr void advance_to(iterator it);

  private:
    iterator current_;                        // exposition only
    sentinel end_;                            // exposition only
    std::locale locale_;                      // exposition only
    basic_scan_args<basic_scan_context> args_; // exposition only
  };
}
```

§ 7.8. Class template `basic_scan_args` synopsis

```
namespace std {
  template<class Context>
  class basic_scan_args {
    size_t size_;                   // exposition only
    basic_scan_arg<Context>* data_; // exposition only

  public:
    basic_scan_args() noexcept;

    template<class... Args>
      basic_scan_args(scan-arg-store<Context, Args...>& store) noexcept;

    basic_scan_arg<Context> get(size_t i) noexcept;
  };

  template<class Context, class... Args>
    basic_scan_args(scan-arg-store<Context, Args...>) -> basic_scan_args<Context>;
}
```

§ 7.9. Concept `scannable`

```
namespace std {
  template<class T, class Context,
           class Scanner = typename Context::template scanner_type<remove_const_t<T>>>
    concept scannable-with =            // exposition only
      semiregular<Scanner> &&
      requires(Scanner& s, const Scanner& cs, T& t, Context& ctx,
               basic_scan_parse_context<typename Context::char_type>& pctx)
      {
        { s.parse(pctx) } -> same_as<expected<typename decltype(pctx)::iterator, scan_error
        { cs.scan(t, ctx) } -> same_as<expected<typename Context::iterator, scan_error>>;
      };

  template<class T, class CharT>
    concept scannable =
      scannable-with<remove_reference_t<T>, basic_scan_context<unspecified, CharT>>;
}
```

§ 7.10. Class template `basic_scan_arg` synopsis

```
namespace std {
  template<class Context>
  class basic_scan_arg {
  public:
    class handle;

  private:
    using char_type = typename Context::char_type;            // exposition only

    variant<
      monostate,
      signed char*, short*, int*, long*, long long*,
      unsigned char*, unsigned short*, unsigned int*, unsigned long*, unsigned long long*,
      bool*, char_type*, void**,
      float*, double*, long double*,
      basic_string<char_type>*, basic_string_view<char_type>*,
      handle> value;                                          // exposition only

    template<class T> explicit basic_scan_arg(T& v) noexcept; // exposition only

  public:
    basic_scan_arg() noexcept;

    explicit operator bool() const noexcept;
  };
}
```

§ 7.11. Exposition-only class template *`scan-arg-store`* synopsis

```
namespace std {
  template<class Context, class... Args>
  class scan-arg-store {                                  // exposition only
    tuple<Args...> args;                                  // exposition only
    array<basic_scan_arg<Context>, sizeof...(Args)> data; // exposition only
  };
}
```

§ References

§ Informative References

[ATTR]
Common Function Attributes. URL: https://gcc.gnu.org/onlinedocs/gcc-8.2.0/gcc/Common-Function-Attributes.html

[BARRY-SO-ANSWER]
Why the standard defines `borrowed_subrange_t` as `common_range`. URL: https://stackoverflow.com/a/66819929

[CODESEARCH]
Andrew Tomazos. *Code search engine website*. URL: https://codesearch.isocpp.org

[FMT]
Victor Zverovich et al. *The fmt library*. URL: https://github.com/fmtlib/fmt

[LWG3081]
Greg Falcon. *Floating point from_chars API does not distinguish between overflow and underflow*. Open. URL: https://wg21.link/lwg3081

[N4412]
Jens Maurer. *N4412: Shortcomings of iostreams*. URL: http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4412.html

[P0355]
Howard E. Hinnant; Tomasz Kamiński. *Extending <chrono> to Calendars and Time Zones*. URL: https://wg21.link/p0355

[P0645]
Victor Zverovich. *Text Formatting*. URL: https://wg21.link/p0645

[P1361]
Victor Zverovich; Daniela Engert; Howard E. Hinnant. *Integration of chrono with text formatting*. URL: https://wg21.link/p1361

[P2286]
Barry Revzin. *Formatting Ranges*. URL: https://wg21.link/p2286

[P2561]
Barry Revzin. *A control flow operator*. URL: https://wg21.link/p2561

[P2637]
Barry Revzin. *Member `visit`*. URL: https://wg21.link/p2637

[PARSE]
Python `parse` package. URL: https://pypi.org/project/parse/

[SCNLIB]
Elias Kosunen. *scnlib: scanf for modern C++*. URL: https://github.com/eliaskosunen/scnlib